

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	40% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	S. Muskan	Reg. No.:	192472182
Section / Batch:		Date:	

Code of Conduct

I, **S. Muskan(192472182)**, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: **S. Muskan**

Instructions

- Read each industry scenario carefully before answering.
- Analyze the given problem from a semantic analysis perspective.
- Provide structured and logical answers with proper justification.
- Support your analysis using examples from the given scenario wherever applicable.
- Use suitable diagrams, semantic representations, logical predicates, workflows, architectures, or tables whenever necessary.
- Clearly state assumptions if additional information is required.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic interpretation of user queries, identify ambiguity in meaning representation, evaluate semantic processing limitations, and improve understanding in banking chatbot applications.	Q1
Syntax-Driven Semantic Analysis	Analyze multiple interpretations of spoken commands, evaluate parsing-related ambiguity, and assess semantic extraction in real-time voice assistant systems.	Q2
First-Order Predicate Calculus, Representing Linguistically Relevant Concepts, Syntax-Driven Semantic Analysis	Design a semantic processing architecture for healthcare reports, model relationships among entities and actions, represent linguistic concepts, and improve semantic extraction accuracy.	Q3

Annexure A – Industry Problem Solving Task

1. A company is developing an AI-powered customer support chatbot for banking services. The chatbot uses Context-Free Grammar (CFG) for parsing user queries. However, it faces issues such as:

Incorrect interpretation of ambiguous sentences

Failure to handle subject–verb agreement

Inefficient parsing for long queries

Example user query:

“Show me the transactions with the card from last month”

a) Analyze the ambiguity present in the given sentence using CFG

b) Evaluate the limitations of CFG in handling real-world conversational queries

c) Evaluate and propose an improved solution using PCFG, feature structures, or Earley parsing

d) Justify how your proposed method improves accuracy and efficiency in an industry setting

2. A voice assistant system (like Alexa/Siri-type applications) processes spoken commands using parsing techniques. The system currently uses top-down parsing, but faces:

Backtracking issues

Poor handling of incomplete or ambiguous input

Difficulty in real-time response

Example command:

“Book a flight to Delhi with a window seat”

a) Analyze how the given sentence can lead to multiple parse structures

b) Analyze the limitations of top-down parsing in this real-time application

c) Analyze how Earley parsing can improve handling of ambiguity and partial inputs

d) Compare and analyze the performance of top-down vs Earley parsing for such industry use cases

3. A healthcare company is developing an AI system to analyze patient medical reports and automatically extract critical information such as diagnosis, prescribed treatments, and follow-up actions.

The system must handle:

Complex and lengthy medical sentences

Ambiguity in sentence structure

Accurate subject-verb agreement

Relationships between medical actions and entities (sub-categorization)

Real-time processing for hospital use

Example input:

"The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai."

The current system, based only on basic Context-Free Grammar (CFG), fails to correctly interpret relationships and produces inconsistent outputs.

a) Design a complete NLP architecture integrating:

CFG for syntactic structure

Probabilistic CFG (PCFG) for ambiguity resolution

Feature Structures for agreement handling

Efficient parsing techniques

b) Develop a step-by-step workflow showing how the system processes the given medical sentence from input to structured output (diagnosis, action, location).

c) Create a method to:

Incorporate sub-categorization frames for medical verbs

Handle real-time data processing

Improve accuracy and scalability in a hospital environment

Q1. AI-Powered Banking Customer Support Chatbot

Given Scenario

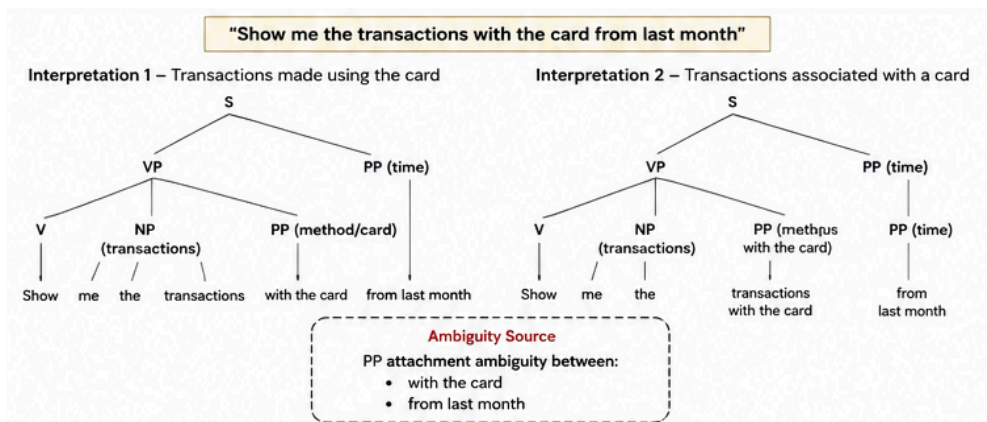
The banking chatbot uses a **Context-Free Grammar (CFG)** to parse customer queries. However, it faces ambiguity, subject–verb agreement problems, and inefficient parsing for long conversational queries.

Example:

“Show me the transactions with the card from last month”

a) Ambiguity Analysis Using CFG

The sentence contains several possible interpretations because the phrases “**with the card**” and “**from last month**” can attach to different parts of the sentence.



Interpretation 1 – Transactions made using the card

The customer wants transactions that were performed using a particular card during the previous month.

Semantic representation:

```
SHOW(  
  TRANSACTIONS(  
    method = CARD,  
    time = LAST_MONTH  
  )  
)
```

Interpretation 2 – Transactions associated with a card

The phrase “with the card” may refer to transactions associated with a particular card, while “from last month” specifies their date.

```
SHOW(  
  TRANSACTIONS(  
    method = CARD,  
    time = LAST_MONTH  
  )  
)
```

```
        associated_card = CARD,  
        date = LAST_MONTH  
    )  
)
```

CFG Representation

A simplified grammar could be:

$S \rightarrow VP$

$VP \rightarrow V NP$

$NP \rightarrow Det N$

$NP \rightarrow NP PP$

$PP \rightarrow P NP$

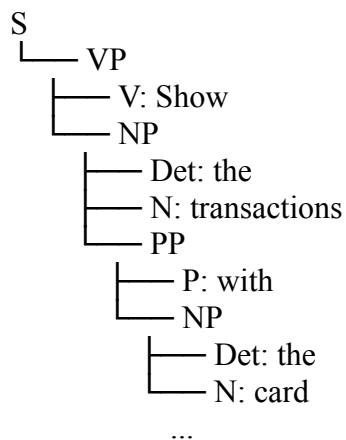
$V \rightarrow \text{Show}$

$Det \rightarrow \text{the}$

$N \rightarrow \text{transactions} \mid \text{card} \mid \text{month}$

$P \rightarrow \text{with} \mid \text{from}$

The parser can generate structures such as:



Another interpretation can attach the prepositional phrases differently.

Main Problem

CFG determines whether a sequence of words is grammatically valid, but it does not inherently determine **which interpretation is most appropriate in the banking context**.

b) Limitations of CFG for Real-World Conversational Queries

Limitation	Effect on Banking Chatbot
Structural ambiguity	Multiple valid parse trees may be produced
No probability	CFG cannot determine the most likely interpretation
No contextual knowledge	“card” may refer to debit, credit, or another card
Weak agreement handling	Subject–verb agreement is difficult without feature information
Large grammar	Conversational language requires many grammar rules
Inefficient long-query parsing	Multiple possible parses increase computational cost
Limited conversational handling	Users may use incomplete or informal sentences
No semantic preference	CFG cannot automatically prefer banking-relevant meanings

For example:

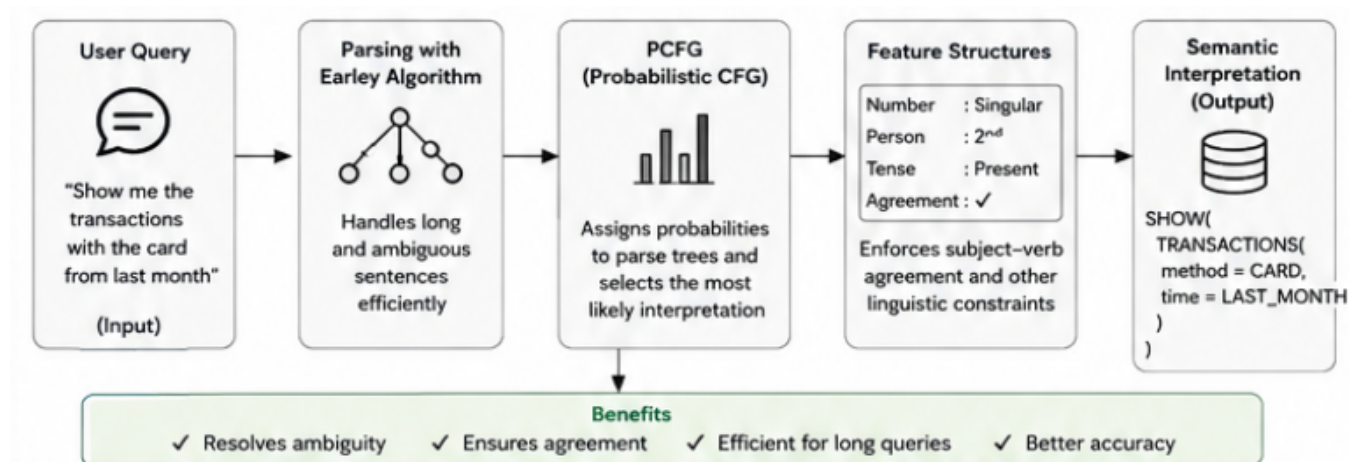
“Show transactions card last month”

A human understands this as a request for card transactions from last month, but a basic CFG may fail because the input does not follow a complete grammatical structure.

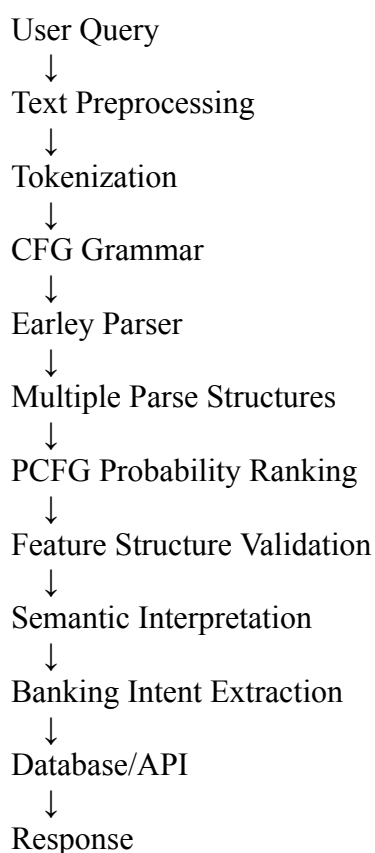
c) Improved Solution Using PCFG, Feature Structures and Earley Parsing

A better solution is to combine:

CFG + PCFG + Feature Structures + Earley Parsing + Semantic Layer



Proposed Architecture



1. PCFG

A **Probabilistic Context-Free Grammar** assigns probabilities to grammar rules.

For example:

$NP \rightarrow NP PP$ [0.30]
 $NP \rightarrow Det N$ [0.70]

$PP \rightarrow P NP$ [1.00]

If historical banking queries show that:

“transactions with the card”

usually means transactions performed using the card, the corresponding parse can receive a higher probability.

Thus:

Parse 1 $\rightarrow$ 0.82

Parse 2 $\rightarrow$ 0.18

The system selects Parse 1.

2. Feature Structures

Feature structures can represent grammatical information such as:

[CAT: NP
NUMBER: PLURAL
PERSON: 3]

For example:

Subject:
[NUMBER: SINGULAR]

Verb:
[NUMBER: PLURAL]

would produce an agreement conflict.

The parser can enforce:

Subject.NUMBER = Verb.NUMBER

This improves grammatical consistency.

3. Earley Parsing

Earley parsing is suitable because it can handle:

- Ambiguous grammars
- General CFGs
- Partial input
- Long sentences

- Incremental parsing

This is useful for conversational banking queries because customers may type incomplete requests such as:

“Show transactions from...”

The parser can maintain partial states instead of completely failing.

d) Justification for Industry Use

The proposed hybrid method improves the banking chatbot in four major ways:

Accuracy

PCFG ranks competing interpretations according to learned probabilities.

Grammatical Correctness

Feature structures ensure agreement constraints are respected.

Efficiency

Earley parsing avoids unnecessary complete backtracking and supports incremental parsing.

Better Banking Semantics

The final semantic layer converts the sentence into a structured banking request.

For example:

User:

"Show me the transactions with the card from last month"

↓

Intent:

VIEW_TRANSACTIONS

↓

Entities:

CARD = specified card

TIME = LAST_MONTH

↓

API:

getTransactions(card, last_month)

↓

Response:

Transactions from the specified card during last month

Conclusion

A basic CFG is useful for establishing syntactic structure, but it is insufficient for real-world banking conversations. A combination of **PCFG, feature structures, and Earley parsing** provides better ambiguity resolution, grammatical validation, and efficient parsing.

Q2. Voice Assistant – Flight Booking

Given Command

“Book a flight to Delhi with a window seat”

The voice assistant currently uses **top-down parsing** and experiences backtracking, ambiguity, and real-time processing problems.

a) Multiple Possible Parse Structures

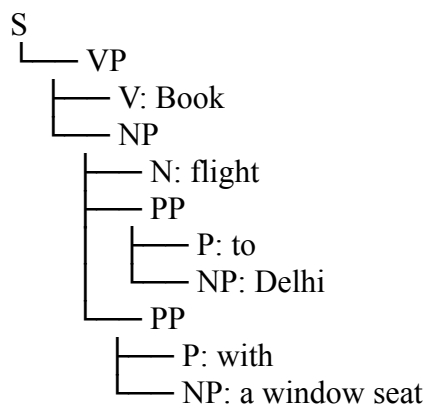
The phrase:

“with a window seat”

can have different syntactic relationships.

Parse 1 – Window seat is a flight-booking preference

This is the most natural interpretation.



Semantic representation:

```
BOOK(
  FLIGHT(
    destination = DELHI,
    seat_preference = WINDOW
```

)
)

Parse 2 – “with a window seat” as an additional flight property

The phrase can also be treated as an attachment to the flight entity:

```
FLIGHT(  
  destination = DELHI,  
  feature = WINDOW_SEAT  
)
```

Although semantically similar, the attachment structure is different.

Semantic Interpretation

The intended meaning can be represented as:

```
BookFlight(  
  destination = Delhi,  
  seat_type = Window  
)
```

The system must therefore identify:

```
Intent = BOOK_FLIGHT  
Destination = DELHI  
SeatPreference = WINDOW
```

b) Limitations of Top-Down Parsing

Top-down parsing starts from the start symbol and attempts to derive the input.

```
S  
↓  
VP  
↓  
V NP  
↓  
...
```

Problem 1 – Backtracking

If an initially selected grammar rule does not match the input, the parser must return to an earlier state and try another rule.

This increases processing time.

Problem 2 – Ambiguity

The phrase:

“with a window seat”

may have multiple possible attachments.

Top-down parsing may explore several alternatives before determining the appropriate structure.

Problem 3 – Incomplete Input

Voice input may arrive incrementally:

“Book a flight...”

followed later by:

“...to Delhi...”

A traditional top-down parser may not efficiently process such partial input.

Problem 4 – Real-Time Requirements

Voice assistants must respond quickly.

Excessive backtracking can increase:

Parsing Time



Response Latency



Poor User Experience

c) Earley Parsing for Ambiguity and Partial Inputs

Earley parsing is particularly suitable for this application because it supports **general CFGs, ambiguity, and incremental/partial parsing**.

Earley Parsing Process

For:

“Book a flight to Delhi with a window seat”

the parser maintains states as words arrive.

Book



Book a



Book a flight



Book a flight to Delhi



Book a flight to Delhi with

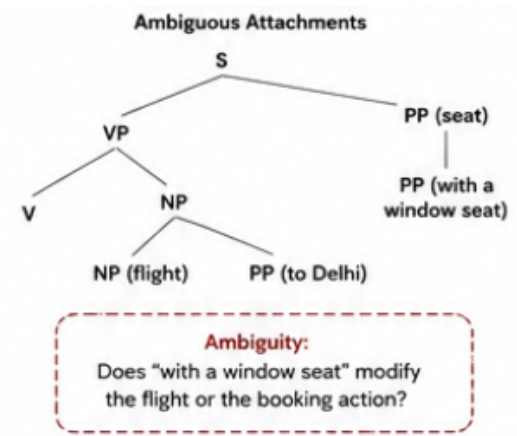
↓

Book a flight to Delhi with a window seat

The parser can maintain possible interpretations without restarting the entire parse.

Earley Parsing Chart (Step Illustration)										
State	Book	a	flight	to	Delhi	with	a	window	seat	\$
0	•	•								
1		•	•	•						
1	•	•	•	•	•					
4			•	•	•	•	•			
5			•	•	•	•	•	•		
6				•	•			•	•	•

- Earley parser keeps multiple states in parallel
- Can handle incomplete input (before \$)
- Ambiguity is preserved until more input arrives



Main Operations

Earley parsing uses:

1. Prediction
2. Scanning
3. Completion

Conceptually:

Prediction

↓

Expected Grammar Rules

↓

Scanning

↓

Input Token

↓

Completion

↓

Completed Constituent

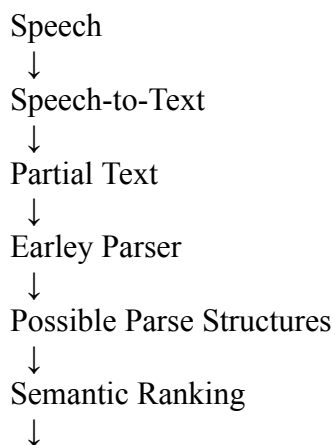
This makes it suitable for spoken commands where input may be incomplete or ambiguous.

d) Top-Down vs Earley Parsing

Feature	Top-Down Parsing	Earley Parsing
Starting point	Start symbol	Input + grammar states
Backtracking	Can be significant	Reduced through chart states
Ambiguity	Difficult to manage efficiently	Handles ambiguous CFGs
Partial input	Poorer support	Better support
General CFG	Depends on implementation	Yes
Real-time processing	Can suffer from latency	More suitable
Repeated work	Possible	Chart stores states
Voice assistant suitability	Limited	High

Criteria	Top-Down Parsing	Earley Parsing
Approach	Starts from start symbol and tries to reach input	Constructs from input, allows all possible states
Backtracking	High (may try many wrong paths)	Low (dynamic chart avoids unnecessary backtracking)
Ambiguity Handling	Poor (often picks one path early)	Good (keeps multiple parses in parallel)
Incomplete Input	Fails or needs full input	Can handle partial / incomplete input
Real-time Suitability	Low (delays due to backtracking)	High (incremental, efficient)
Use in Industry	Simple grammars	Complex, ambiguous, real-time systems

Industry Workflow



Intent Extraction



Flight Booking API



Confirmation

Final Semantic Output

Intent:

BOOK_FLIGHT

Destination:

Delhi

Seat Preference:

Window

Conclusion

Top-down parsing is simple but can become inefficient when ambiguity and incomplete speech are present. **Earley parsing is more appropriate for real-time voice assistants** because it can handle ambiguous CFGs and partial input while reducing unnecessary re-parsing.

Q3. Healthcare Medical Report Analysis System

Given Sentence

“The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai.”

The healthcare system must extract:

- Diagnosis
- Prescribed treatment/action
- Follow-up action
- Location
- Relationships between entities and actions

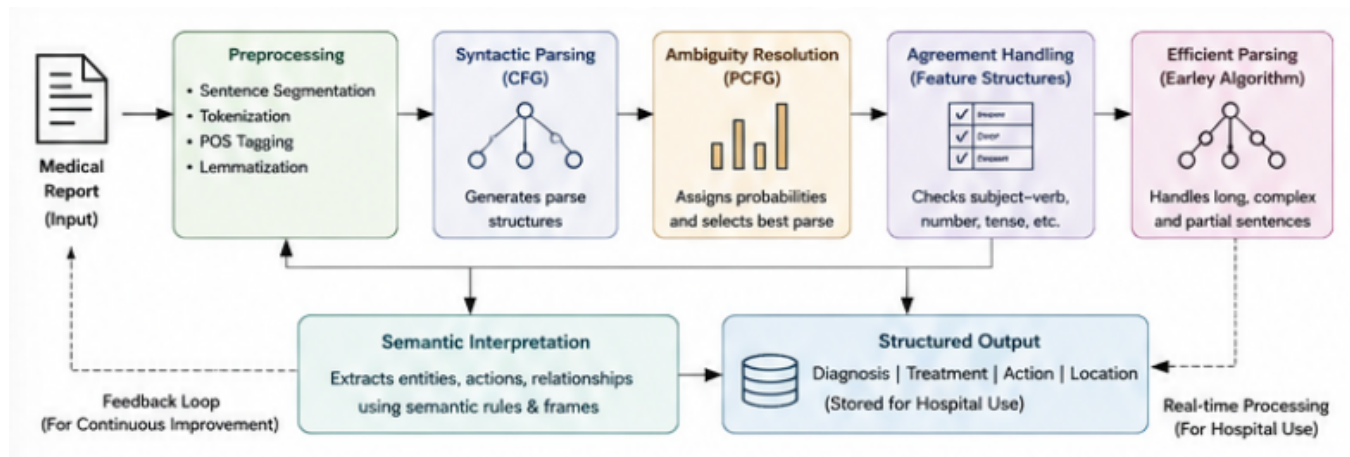
a) Complete NLP Architecture

The proposed architecture combines:

- CFG
- PCFG

- Feature Structures
- Efficient Parsing
- Semantic Analysis
- Medical Entity Recognition
- Sub-categorization Frames
- Structured Output

Architecture Diagram



CFG Layer

CFG determines the basic syntactic structure.

A simplified grammar is:

$S \rightarrow NP VP$

$NP \rightarrow Det N$

$NP \rightarrow Det N RelClause$

$VP \rightarrow V NP$

$VP \rightarrow V GerundPhrase$

$VP \rightarrow V NP Conj GerundPhrase$

$RelClause \rightarrow RelPronoun VP$

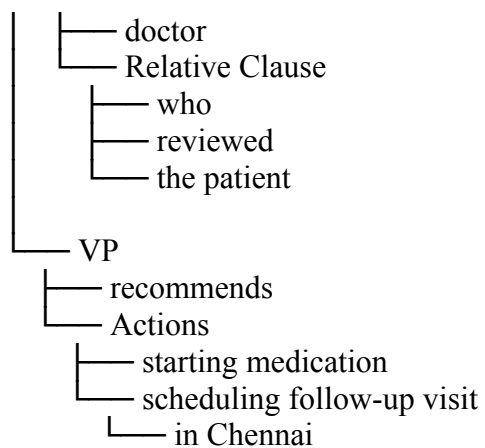
$PP \rightarrow P NP$

The sentence can therefore be decomposed as:

S

```

  |
  +--- NP
  |   |
  |   +--- The
  
```



PCFG Layer

The sentence contains structural complexity.

The system may generate multiple candidate interpretations.

PCFG assigns probabilities:

Parse A $\rightarrow P = 0.88$

Parse B $\rightarrow P = 0.12$

The highest-probability syntactic structure is selected.

This helps resolve attachment ambiguity, especially involving:

“last week”

and

“in Chennai”.

Feature Structure Layer

Feature structures ensure agreement.

Example:

Doctor:

[CAT: NP

NUMBER: SINGULAR

PERSON: 3]

Verb:

Recommends:

[CAT: V

NUMBER: SINGULAR
PERSON: 3]

Constraint:

Doctor.NUMBER = Recommends.NUMBER

Therefore:

The doctor recommends

is valid.

An incorrect structure such as:

The doctor recommend

would violate the agreement feature.

b) Step-by-Step Workflow

Step 1 – Input

"The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai."

Step 2 – Tokenization

The system separates the sentence into tokens:

The | doctor | who | reviewed | the | patient |
last | week | recommends | starting | medication |
and | scheduling | a | follow-up | visit | in | Chennai

Step 3 – Syntactic Parsing

CFG identifies grammatical constituents.

Subject = The doctor who reviewed the patient last week

Main Verb = recommends

Actions =

starting medication

scheduling a follow-up visit in Chennai

Step 4 – PCFG Ambiguity Resolution

PCFG evaluates possible structures.

The system selects the most probable interpretation based on learned medical-language patterns.

Step 5 – Feature Structure Validation

The parser verifies grammatical agreement.

Subject:

doctor

Number = singular

Verb:

recommends

Number = singular

Result:

VALID

Step 6 – Medical Entity Extraction

Important entities are identified:

Entity	Type
doctor	Medical professional
patient	Patient
medication	Treatment
follow-up visit	Medical action
Chennai	Location

Step 7 – Action Extraction

The main medical action is:

recommends

Recommended actions:

starting medication
scheduling follow-up visit

Step 8 – Relationship Extraction

The system identifies:

DOCTOR
└── recommends ──→ MEDICATION

DOCTOR
└── recommends ──→ FOLLOW-UP VISIT

FOLLOW-UP VISIT
└── location ──→ CHENNAI

Step 9 – Structured Output

```
{  
  "subject": "doctor",  
  "action": "recommends",  
  "treatment": "starting medication",  
  "follow_up": "scheduling a follow-up visit",  
  "location": "Chennai"  
}
```

Important Assumption

The sentence does **not explicitly state a diagnosis**. Therefore, the system should not invent one.

Diagnosis = Not specified

This is important in a healthcare system because unsupported medical information must not be generated.

First-Order Predicate Calculus Representation

The sentence can be represented using predicates.

Let:

Doctor(x)
Patient(y)
Medication(z)
FollowUpVisit(v)
Location(v, Chennai)
Reviewed(x, y, LastWeek)
Recommends(x, z)
Recommends(x, v)

Then:

$\exists x \exists y \exists z \exists v$
[
 Doctor(x)
 $\wedge$ Patient(y)
 $\wedge$ Reviewed(x,y,LastWeek)
 $\wedge$ Medication(z)
 $\wedge$ Recommends(x,z)
 $\wedge$ FollowUpVisit(v)
 $\wedge$ Recommends(x,v)
 $\wedge$ Location(v,Chennai)
]

This represents the relationships between the doctor, patient, treatment, follow-up visit, and location.

c) Proposed Method for Sub-Categorization, Real-Time Processing and Scalability

1. Sub-Categorization Frames for Medical Verbs

Medical verbs have characteristic argument structures.

For example:

recommend
 Subject: Medical professional
 Object/Complement: Treatment or medical action

Frame:

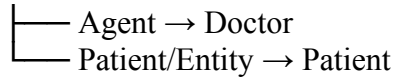
RECOMMEND
├── Agent → Doctor
└── Action → Medication / Follow-up

For:

review

the frame can be:

REVIEW

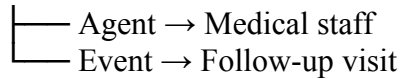


For:

schedule

the frame can be:

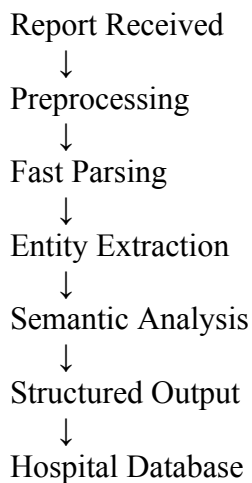
SCHEDULE



These frames help the system distinguish who performs an action and what the action affects.

2. Real-Time Data Processing

For hospital environments, processing should be organized as a pipeline:



Efficient parsing techniques such as **Earley parsing with chart/state reuse** can reduce unnecessary repeated computation.

Priority can be given to important entities such as:

Diagnosis
Medication
Allergy
Procedure
Follow-up
Doctor
Patient
Location

3. Accuracy Improvement

A hybrid architecture can improve accuracy:

CFG



Syntactic Structure

PCFG



Ambiguity Resolution

Feature Structures



Agreement Validation

Sub-Categorization Frames



Verb-Entity Relationships

Medical NER



Medical Entity Identification

Semantic Parser

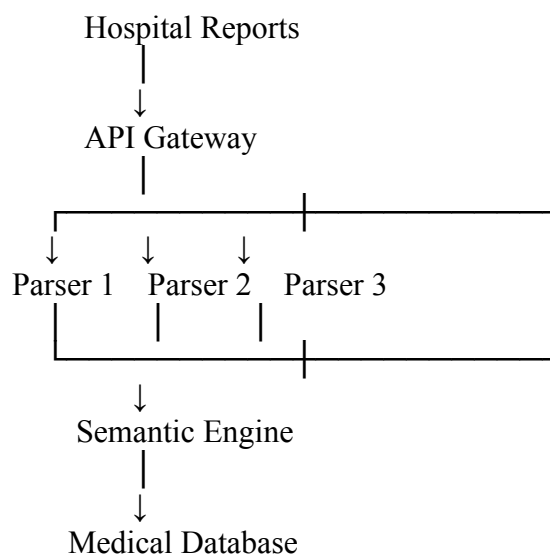


Structured Meaning

Multiple layers provide validation instead of depending on one grammar alone.

4. Scalability in a Hospital Environment

A scalable implementation can use:



Multiple processing workers can handle reports simultaneously.

Caching commonly used grammar structures and medical terminology can further reduce processing time.

Overall Comparison

Requirement	Basic CFG	Proposed Hybrid System
Syntactic analysis	Yes	Yes
Ambiguity handling	Limited	PCFG
Agreement handling	Limited	Feature Structures
Long sentences	Limited	Efficient parsing
Medical entities	Limited	Medical NER
Verb relationships	Limited	Sub-categorization
Partial input	Limited	Earley/incremental parsing
Probability	No	PCFG
Real-time processing	Limited	Optimized pipeline
Scalability	Limited	Parallel/service architecture

Final Conclusion

The three industry scenarios demonstrate that **basic CFG alone is insufficient for complex real-world NLP systems**.

For banking, combining **CFG, PCFG, feature structures, and Earley parsing** improves interpretation of ambiguous conversational queries.

For voice assistants, **Earley parsing** is better suited to ambiguous and incomplete spoken commands because it maintains parsing states and reduces unnecessary backtracking.

For healthcare, a complete semantic-processing architecture should combine **CFG, PCFG, feature structures, efficient parsing, medical entity extraction, sub-categorization frames, and predicate-based semantic representation.**

The resulting architecture transforms raw language into structured meaning while improving:

Accuracy

+

Ambiguity Resolution

+

Grammatical Consistency

+

Real-Time Performance

+

Scalability

↓

Reliable Industry NLP System

Assumption: Where the given scenarios do not explicitly provide a diagnosis, medical condition, card type, or other entity, the system should mark the information as **not specified** rather than inventing it.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q. No. / Criteria Level	Understanding of Industry Problem	Application of Engineering Concepts	Solution Design	Use of Tools	Analysis	Professionalism	Sub. Total	Total %
1								
2								
3								

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____